

MOBILE APPLICATION DEVELOPMENT

WEEK 1 PROJECT PROPOSAL: SPENDWISE PROJECT

SpendWise is a personal finance mobile application developed for Android users to help them manage and keep track of their financial activities. The system allows users to record their expenses, set monthly spending limits, monitor their daily spending, and review their overall financial situation through generated reports. The application uses Java for Android development, with a local Room SQLite database for storing data on the device. It also connects to a RESTful API and a central MySQL database for managing data online.

The project is being developed by Group 4, consisting of Naomi Akilimali and Rabbecca Chola. The application will use the MVVM architecture, secure user authentication, charts for displaying financial information, file export features, and synchronization between offline and online data.

Problem Statement

Managing personal finances can be difficult when people do not have a proper way of recording their income and expenses. Without organized records, users may find it difficult to understand their spending habits and may end up spending more than they planned. It can also be difficult to notice spending problems without clear warnings and visual information. Another challenge occurs when there is no internet connection, as some financial applications may not allow users to continue recording their expenses. Understanding long term spending can also be difficult without proper summaries and charts.

SpendWise aims to address these problems by providing one mobile application where users can record their financial activities both offline and online. Users will be able to organize expenses into categories, set monthly spending limits, receive spending warnings, view financial information using charts, and synchronize their local data with the online server.

Project Aim

The main aim of this project is to design, develop, and document a secure Java based Android personal finance application. The system will allow users to manage their income and expenses, set budgets, use the application without an internet connection, synchronize information with an online server, and view financial reports.

To achieve this aim, the project will provide secure user registration and login, token based authentication, and protected user information. Users will be able to add, view, edit, and delete income and expense records. The system will also allow users to create monthly budgets and monitor their spending against those budgets. A local Room database will allow users to save transactions while offline and synchronize them with the online server when the internet connection becomes available. The system will also provide financial charts, CSV and PDF report exports, and will follow the Model View ViewModel (MVVM) architecture.

Stakeholder Analysis

The main stakeholders of the system are the end users, the development team, and the academic evaluator. End users need an application that is easy to use, fast, and able to work without an internet connection. They will use the system to record financial activities, manage budgets, and generate reports.

The development team consists of Naomi Akilimali and Rabecca Chola. The team will be responsible for developing the mobile application, designing the user interface, creating the backend and database, testing the system, and preparing the project documentation.

The academic evaluator will assess the technical development of the system, the quality of the architecture, the project documentation, the GitHub commit history, and the final demonstration of the application.

System Requirements

Functional Requirements

The application will allow users to register, log in securely, and manage their profiles. Users will be able to add, view, update, and delete income and expense records. They will also be able to organize their expenses into different categories.

The system will allow users to set monthly spending limits and will calculate the remaining amount in each budget. It will also provide warnings when users reach or go beyond their spending limits.

For financial analysis, the system will provide monthly summaries, category breakdowns, and financial charts. The application will also work without an internet connection by storing transactions locally using Room and SQLite. When the internet connection becomes available, the stored information will be synchronized with the remote MySQL database through the REST API. Users will also be able to export financial reports in CSV or PDF format.

Non Functional Requirements

The application will use Material Design principles to provide a clean and easy to use interface. Common operations such as budget calculations and displaying charts should respond within a reasonable amount of time.

The system will use an offline first approach so that important functions can continue working without an internet connection. Security will be improved by protecting passwords and using JWT authentication when communicating with the API. The MVVM architecture will also be used to separate the user interface, application logic, and database operations.

Prioritised Feature List

The main system features are organized using the MoSCoW prioritization method. The SpendWise system architecture is organized to prioritize core functional capabilities during development. Most importantly the infrastructure components focus on secure user authentication via the JSON Web Token, full CRUD logic for income and expenses, a dynamic category budgeting engine, local Room database persistence with an offline queue, and REST API integration for server synchronization. Analytical and reporting capabilities enhance functionality through visual financial charts, downloadable CSV or PDF summary exports, and local budget alert notifications. Finally, optional enhancements introduce visual personalization through custom category icon selections.

System Architecture

The SpendWise system is structured as a multi-tier platform connecting a Java Android client, a RESTful API backend, and a remote MySQL database. Users interact with the mobile application through a Material Design interface built around the Model View View Model (MVVM) pattern. To provide an offline first experience, user inputs are stored directly in a local

Room SQLite database before any network transport occurs. When an active network connection is detected, the repository layer uses Retrofit to send JSON payloads to the backend API. The API validates incoming requests via JWT tokens, executes domain logic, and commits records to the central MySQL database, guaranteeing data consistency across devices.

Use Case and Interaction Analysis

The main user of the system is the End User. When recording a transaction, the user will enter information such as the amount, category, date, and description. The application will check the information and save it in the local database. It will then update the user's budget and prepare the transaction for synchronization with the server.

When setting a budget, the user will enter a monthly spending limit. The system will keep track of the user's expenses and compare them with the set limit. When the user approaches or exceeds the limit, the system will provide a warning.

The synchronization process will take place in the background. When an internet connection becomes available, the application will send stored transactions to the server. The server will process the information, and the application will update its local records to match the online database.

Team Roles and Individual Responsibility Matrix

The responsibilities are divided between the two team members across the frontend, backend, database, testing, and documentation.

Naomi Akilimali

UI and Frontend: Android XML Layouts, Activities, Fragments, Material Design Styling, and Navigation Graph setup.

Logic and Architecture: User authentication flows, ViewModel state handling, and budget calculation rules.

Data and Backend: REST API endpoints, Remote MySQL Database schema design, and server environment configuration.

Quality and Docs: Business logic Unit Testing, API integration testing, System Architecture diagrams, and Proposal documentation.

Rabecca Chola

UI and Frontend: Form validation UI, RecyclerViews, Custom Adapters, Dialogs, and usability refinement.

Logic and Architecture: Search and filtering algorithms, Transaction ViewModel, and MPAndroidChart integration.

Data and Backend: Room DAO implementations, SQLite local entities, and Network Synchronization Manager.

Quality and Docs: UI Instrumentation testing, offline edge case testing, ER Diagrams, User Manual preparation, and APK packaging.

GitHub Repository and Development Tracking

The project source code, API files, database files, and technical documentation will be stored on GitHub. GitHub will allow the team to work together and keep a record of the project's development progress.

Repository URL:

<https://github.com/NaomiLeya22/SpendWise.git>